

Vodafone Customer Platform

Project Story & Business Case

1. The Story Behind the Project

Today, telecom customers expect to access their services quickly and reliably without depending on customer support for every request.

This creates an important challenge for telecom companies:

How can we provide a simple customer experience while building a platform that can handle increasing traffic, remain available, and protect customer information?

Based on this idea, I designed a **Cloud-Based Customer Platform on Microsoft Azure** as a Proof of Concept.

The goal was not simply to build a website. The goal was to design the **cloud infrastructure behind a telecom customer service** and demonstrate how it could scale, remain available, and be securely operated.

The customer sees a simple interface where they can retrieve their profile, mobile plan, and balance.

Behind this simple experience, the cloud platform handles:

Security → Traffic Management → Scalability → Application Processing → Private Connectivity → Database Resilience → Monitoring → Automation

This is the main story of the project.

2. The Business Problem

Imagine a Vodafone customer trying to access a digital service during a period of high traffic.

If the application depends on a single server, that server can become overloaded or unavailable.

If the database becomes unavailable, the application may also stop working.

If sensitive credentials are stored directly inside the application, security risks increase.

And if infrastructure is manually configured, maintaining multiple environments becomes difficult.

Therefore, the platform needs to answer five main questions:

1. **How do we handle increasing traffic?**
2. **How do we keep the application available?**
3. **How do we protect the customer-facing service?**
4. **How do we protect customer data and internal communication?**
5. **How do we deploy and manage the infrastructure efficiently?**

The project was designed around these requirements.

3. The Proposed Solution

I designed an Azure-based architecture where each layer has a specific responsibility.

The customer first reaches **Azure Application Gateway with WAF**, which acts as the secure public entry point.

Traffic is then distributed to a **Virtual Machine Scale Set**, where multiple application servers can run simultaneously.

The application uses **Nginx and Node.js/Express** to provide the customer APIs.

Customer information is stored in **Azure SQL Database**, with a secondary SQL server in another region to provide database resilience.

Sensitive credentials are stored in **Azure Key Vault** and accessed using **Managed Identity**.

Azure SQL, Storage, and Key Vault are accessed through **Private Endpoints and Private DNS**, allowing the application to communicate with these services privately through the Azure Virtual Network.

The infrastructure is deployed and managed using **Terraform**.

Monitoring is provided through **Azure Monitor, Log Analytics, and Application Insights**.

Simplified Architecture

Customer

↓

Application Gateway + WAF

↓

VM Scale Set

↓

Nginx + Node.js API

↓

Private Network Connectivity

↓

Azure SQL

Alongside the architecture:

Key Vault + Storage + Private Endpoints + Private DNS + Monitoring + Terraform

This architecture separates the application into different layers, making it easier to scale, secure, monitor, and maintain.

4. How the Customer Experience Works

From the customer's perspective, the process is simple.

The customer opens the platform and enters their customer ID.

The request reaches the Application Gateway.

The WAF provides an additional security layer before the request reaches the application.

The Application Gateway forwards the request to a healthy VM instance inside the VM Scale Set.

Nginx receives the request and forwards the API call to the Node.js backend.

The backend uses its Managed Identity to securely access the required secrets from Azure Key Vault.

The connection to Key Vault is established through its **Private Endpoint**, rather than relying on public access.

The backend then communicates with Azure SQL through the SQL **Private Endpoint**.

The database returns the customer information, and the result is sent back through the application to the customer.

Customer Journey

Customer → Application Gateway/WAF → VMSS → Nginx → Node.js API → Private Endpoint → Azure SQL → Customer

The important point is that the customer experiences a simple service, while the cloud platform handles the complexity behind it.

5. Scalability and Availability

A telecom platform cannot depend on a single application server.

The web application therefore runs on an Azure Virtual Machine Scale Set with multiple instances.

The initial configuration uses **two instances** and can scale up to **five instances**.

Autoscaling is configured according to CPU utilization:

- CPU above 70% → Scale Out.
- CPU below 30% → Scale In.
- Minimum instances: 2.
- Maximum instances: 5.

This allows the platform to respond automatically to changes in traffic.

The Application Gateway also uses health probes to identify healthy backend instances and route traffic accordingly.

If one application instance becomes unhealthy, traffic can be directed toward healthy instances.

This provides a foundation for **horizontal scalability and high availability**.

6. Security by Design

Security was considered as part of the architecture rather than added at the end.

The platform uses multiple security layers.

6.1 Public Application Protection

Application Gateway WAF protects the public entry point and provides protection against common web attacks.

6.2 Network Security

Network Security Groups control communication between the different subnets.

For example, database traffic is restricted to the required application network instead of allowing unrestricted access.

6.3 Private Connectivity

The project uses **Azure Private Endpoints** for:

- Azure SQL
- Azure Storage Blob
- Azure Key Vault

These services are accessed through private IP addresses inside the Azure Virtual Network.

Private DNS Zones are used so that the normal Azure service names can resolve to the private endpoints.

This reduces the dependency on public network access for internal application communication.

6.4 Secret Management

Database credentials are stored in **Azure Key Vault** rather than being directly embedded in the application.

The VM Scale Set uses **Managed Identity** to retrieve the required secrets.

6.5 Database Protection

Azure SQL public network access is disabled.

The application communicates with SQL through the private network and its Private Endpoint.

6.6 Encryption

TLS is used to protect communication with Azure services.

Together, these controls provide a **defense-in-depth security architecture**.

7. Database Resilience

For a telecom service, the database is one of the most critical components.

If the application is running but the database is unavailable, the customer service can still be affected.

To address this, the project uses:

Primary Azure SQL → Secondary Azure SQL → Failover Group

The primary database is hosted in **Central US**, while the secondary SQL server is hosted in **UK West**.

An Azure SQL Failover Group is configured between them to provide a mechanism for database failover if the primary database becomes unavailable.

At the same time, the primary SQL server is accessed privately through an Azure Private Endpoint.

This combines two important concepts:

Database Resilience + Private Database Connectivity

For a real Vodafone production environment, the final regions, RTO, RPO, data residency requirements, and failover strategy would be determined according to Vodafone's production requirements.

8. Private Access to Azure Services

An important part of the architecture is that internal Azure services are not treated as public resources.

The project creates private connectivity for three major services.

Azure SQL

VMSS → SQL Private Endpoint → Azure SQL

Used for customer and application data.

Azure Key Vault

VMSS → Key Vault Private Endpoint → Key Vault

Used for secure retrieval of database credentials.

Azure Storage

Application → Storage Private Endpoint → Blob Storage

Used for private application storage.

Private DNS zones are linked to the Virtual Network so that the applications can resolve the Azure service names correctly through their private endpoints.

This gives the platform a more enterprise-oriented network design.

9. Application Implementation

The customer platform consists of a simple frontend and backend API.

The frontend allows a customer ID to be entered and retrieves the customer's profile.

The backend is implemented using **Node.js and Express**.

The API provides functions such as:

- Application health check.
- Customer profile lookup.

The application is hosted on the VM Scale Set.

Nginx serves the frontend and works as a reverse proxy for the backend API.

The database contains customer and plan information.

For the PoC, test data is generated to simulate a larger customer base, allowing the infrastructure and application to be tested without using real customer data.

10. Monitoring and Operations

A cloud platform should not only run; the operations team should also be able to understand what is happening.

The project therefore includes:

- Azure Monitor
- Log Analytics
- Application Insights
- VMSS CPU monitoring
- Application health checks

A CPU alert is configured to detect high resource utilization.

The Application Gateway also uses the application health endpoint to determine whether backend instances are healthy.

This creates a basic observability layer that can help identify infrastructure or application problems.

11. Infrastructure Automation with Terraform

Another important part of the project is **Infrastructure as Code**.

Instead of manually creating every Azure resource through the Portal, Terraform defines the infrastructure as code.

The Terraform configuration manages:

- Resource Group
- Virtual Network and Subnets
- Network Security Groups
- Application Gateway
- WAF
- VM Scale Set
- Azure SQL
- SQL Failover Group
- Private Endpoints
- Private DNS Zones
- Storage
- Key Vault
- Monitoring
- Autoscaling

This makes the infrastructure **repeatable, consistent, and easier to maintain.**

The same architecture can be recreated for:

Development → Testing → Production

without manually rebuilding every resource.

12. Business Value for Vodafone

The main value of the project is not simply creating a website.

The project demonstrates a reusable **cloud architecture pattern for a customer-facing telecom service.**

Potential benefits include:

- Scalability during high traffic.
- Improved application availability.
- Secure public application access.
- Private communication with Azure services.
- Database resilience.
- Centralized secret management.
- Infrastructure monitoring.
- Automated infrastructure deployment.

- Reduced manual configuration.
- Easier replication across environments.

The important point is that the architecture can be used as a foundation rather than being limited to the current demonstration application.

13. How This Could Evolve into a Vodafone Platform

The current project is intentionally simplified because it is a training Proof of Concept.

In a real Vodafone environment, the platform could be integrated with existing enterprise systems and APIs.

Possible future capabilities include:

- Customer authentication and identity management.
- Mobile usage information.
- Billing information.
- Available plans and offers.
- SIM/eSIM services.
- Payment services.
- Customer support.
- Notifications.
- Service management.

The customer profile API used in the PoC could eventually communicate with real Vodafone backend systems instead of demonstration data.

The architecture could therefore evolve from a simple customer-profile application into a broader digital customer platform.

Future Architecture

Customer

↓

Identity & Authentication

↓

Application Gateway + WAF

↓

Scalable Application Layer

↓

API Layer

↓

Vodafone Enterprise Systems

With:

Private Connectivity + Key Vault + Monitoring + Database Resilience + Security

supporting the complete platform.

14. From Proof of Concept to Production

The current implementation is intentionally a **Proof of Concept**, not a production Vodafone system.

The project already demonstrates several production-oriented concepts such as:

- Horizontal scaling.
- WAF protection.
- Private Endpoints.
- Private DNS.
- Managed Identity.
- Key Vault.
- Database failover.
- Monitoring.
- Infrastructure as Code.

Before production deployment, additional enterprise requirements would need to be addressed, including:

- Enterprise Identity and Access Management.
- Production certificate lifecycle management.
- CI/CD pipelines and approval processes.
- Centralized security monitoring/SIEM.
- Vulnerability management.
- Backup and restore testing.
- Disaster recovery testing.

- Load and performance testing.
- Cost optimization.
- SLA/SLO definition.
- Compliance and data residency.
- Incident response procedures.

These requirements would be aligned with Vodafone's internal architecture, security, compliance, and operational standards.

15. The Message to Management

When presenting the project, the focus should be on the **business problem and the solution**, not on listing Azure services.

A suitable opening is:

“The project I built is not only a customer website. I treated the website as a telecom service and designed the cloud platform required behind it.”

The customer gets a simple digital experience.

Behind that experience, the platform provides:

Secure Access

↓

Scalable Application

↓

Private Connectivity

↓

Resilient Database

↓

Centralized Secrets

↓

Monitoring

↓

Infrastructure Automation

The architecture demonstrates how cloud engineering can provide a reliable foundation for customer-facing telecom services.

16. Final Vision

The final vision is to take this concept beyond a training website and connect it to real telecom business services.

Current PoC

Customer → Azure Cloud Platform → Demonstration Database

Future Vodafone Platform

Customer → Identity → Secure Cloud Platform → APIs → Vodafone Enterprise Systems

The cloud platform becomes the foundation connecting the customer experience with Vodafone's existing services.

17. Conclusion

This project demonstrates a complete cloud architecture for a customer-facing telecom application.

The main focus was not on using the largest number of Azure services, but on solving real engineering problems:

- How to scale?
- How to stay available?
- How to secure the application?
- How to keep internal services private?
- How to protect customer data?
- How to monitor the platform?
- How to automate infrastructure?

The result is a reusable architecture combining:

Scalability + Security + Private Connectivity + Availability + Database Resilience + Monitoring + Automation

The next step would be to validate this architecture against Vodafone's real business, security, performance, and compliance requirements and integrate it with actual Vodafone customer and service APIs.

Project Story in One Sentence

“I designed a secure and scalable Azure-based customer platform that demonstrates how Vodafone could provide a reliable self-service experience using automated infrastructure, horizontal scaling, protected application access, private connectivity, database resilience, and centralized monitoring.”